FH Salzburg
MultiMediaTechnology

Basics Game Development

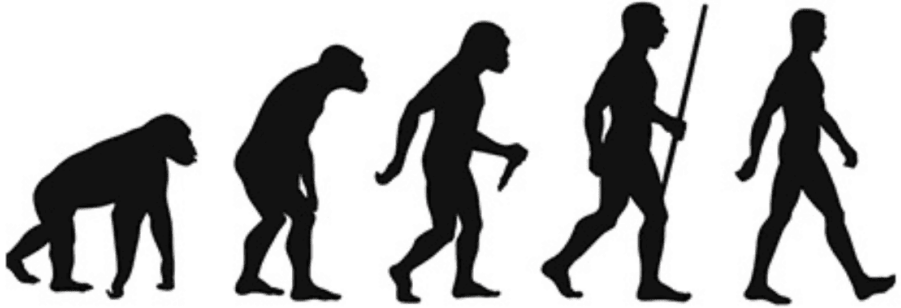
Transformations - Lab

Technology
Health
Media

10 Minute Quiz

Today's Topics

- Refactoring (Game Class)
- Gameloop & DeltaTime
- Lerp



Refactoring

Improving the Design of Existing Code

Refactoring – Game Class

Where we probably are

```
static void Main(string[] args)
{
    VideoMode mode = new VideoMode(800, 600);
    RenderWindow window = new RenderWindow(mode, "My window");

    window.Closed += (sender, args) => window.Close();

    while (window.IsOpen)
    {
        window.DispatchEvents();
        window.Clear(Color.Blue);
        window.Display();
    }
}
```

What we probably want

```
0 references
internal class Program
{
    0 references
    private static void Main(string[] args)
    {
        var game = new Game( );
        game.Run();
    }
}
```

Refactoring – Game Class - Constructor

```
1 reference
public Game()
{
    VideoMode mode = new(WIDTH, HEIGHT);
    window = new RenderWindow(mode, TITLE);

    // This is an Event Callback
    window.Closed += CloseGame;
}

1 reference
private void CloseGame(object? sender, EventArgs e)
{
    // Collect Data ( Highscore )
    // Save to Savegame
    // Send to Server
    // etc

    window?.Close();
}
```

```
    window.Resized += OnWindowResized;

private static void OnWindowResized(object? sender, SizeEventArgs e)
{
    view.Size = (Vector2f)window.Size;
    window.SetView(view);
}
```

Refactoring – Game Class – Initialize()

```
internal class Game
{
    1 reference
    public void Run()
    {
        Initialize();

        while (window.IsOpen)
        {
            var deltaTime = clock.Restart().AsSeconds();

            HandleEvents();

            Update(deltaTime);

            Draw();
        }
    }
```

```
1 reference
private void Initialize()
{
    // Setup Stuff
    // that only needs to be done once

    // Like load Texture and create the player Sprite
    playerTexture = new Texture(@".\Assets\car.png");
    playerSprite = new Sprite(playerTexture);

    // or View
    view = new View(playerSprite.Position, (Vector2f)window.Size);

    // or Generate your World
    // or anything else
}
```

Refactoring – Game Class – HandleEvents()

```
internal class Game
{
    1 reference
    public void Run()
    {
        Initialize();

        while (window.IsOpen)
        {
            var deltaTime = clock.Restart().AsSeconds();

            HandleEvents();

            Update(deltaTime);

            Draw();
        }
    }
```

```
1 reference
private void HandleEvents()
{
    window.DispatchEvents();
}
```

https://www.sfml-dev.org/tutorials/2.5/window-events.php

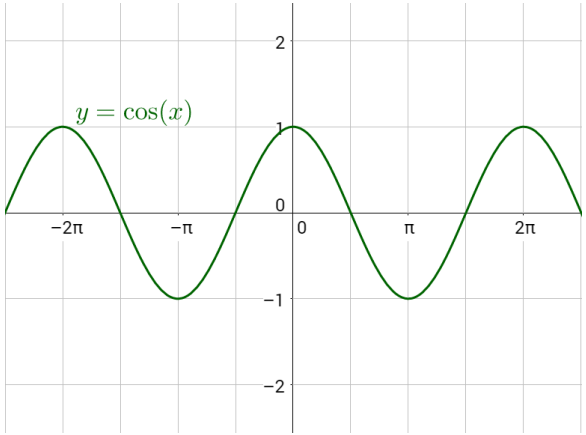
Refactoring – Game Class – Update()

```
internal class Game
{
    1 reference
    public void Run()
    {
        Initialize();

        while (window.IsOpen)
        {
            var deltaTime = clock.Restart().AsSeconds();

            HandleEvents();

            Update(deltaTime);

            Draw();
        }
    }
```

```
1 reference
private void Update(float deltaTime)
{
    // Example move player Sprite
    playerSprite.Position += moveVector * deltaTime;
}
```

Refactoring – Game Class – Update()

```
1 reference
private void Update(float deltaTime)
{
    // Example Move Sprite Y with time-based Sin
    gameTime += deltaTime;
    playerSprite.Position = new Vector2f(playerSprite.Position.X, MathF.Sin(gameTime) * 100 );
}
```

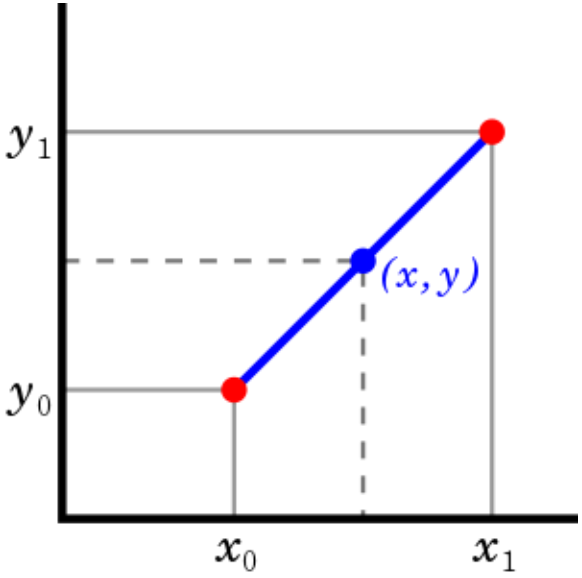


Lerp – Linear Interpolation

```
public static float Lerp(float start, float target, float t)
{
    return start + (target - start) * t;
}
```

```
float progress = Math.Clamp(gameTime / 2f, 0, 1);

float newPosition = Utils.Lerp(START_POSITION.X, targetPosition, progress);
playerSprite.Position = new Vector2f(newPosition, playerSprite.Position.Y);
```

Note: Player speed is now dependent both on gameTime, as well as distance between start and target



Refactoring – Game Class – Draw()

```
internal class Game
{
    1 reference
    public void Run()
    {
        Initialize();

        while (window.IsOpen)
        {
            var deltaTime = clock.Restart().AsSeconds();

            HandleEvents();

            Update(deltaTime);

            Draw();
        }
    }
```

```
1 reference
private void Draw()
{
    window.Clear(Color.Blue);

    window.SetView(view);

    window.Draw(playerSprite);

    window.Display();
}
```

FH Salzburg
MultiMediaTechnology

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at